



Module 11-1

Using Verilog Operators

cādence®

This module provides an explanation and example for each Verilog operator.

Module Objective

In this module, you:

- Choose and use the Verilog operators correctly

Operators

Category	Symbol
Bit-wise	~ & ^ ~^
Reduction	& ~& ~ ^ ~^ ^~
Logical	! &&
Arithmetic	** * / % + -
Shift	<< >> <<< >>>
Relational	< > <= >=
Equality	== != === !==
Conditional	? :
Concatenation	{ }
Replication	{ { } }



This page does not contain notes.

Bit-Wise Operators

not	~
and	&
or	
xor	^
xnor	~^
xnor	^~

- Bit-wise operators operate on vectors.
- Operations are performed bit by bit on individual bits.
- Unknown bits in an operand do not necessarily lead to unknown bits in the result.

```
module bitwise;
  reg [3:0] rega, regb, regc;
  reg [3:0] num;

  initial
    begin
      rega = 4'b1001;
      regb = 4'b1010;
      regc = 4'b11x0;

      num = ~rega;           // num = 0110
      num = rega & 0;        // num = 0000
      num = rega & regb;     // num = 1000
      num = rega | regb;     // num = 1011
      num = regb & regc;     // num = 10x0
      num = regb | regc;     // num = 1110
    end
endmodule
```

The bitwise operators perform logical operations in a bitwise manner.

The bitwise unary negation operator inverts the logical sense of each bit of its operand. Each 0 becomes 1 and each 1 becomes 0, and each high-impedance bit becomes unknown.

The bitwise binary operators first zero-extend a smaller operand to match the width of a larger operand, and then perform logical operations on individual bit positions. Depending upon the operation, bits in one operand that are 0 or 1 can mask bits in the same position of the other operand that are high-impedance or unknown, so unknown bits in an operand do not necessarily produce unknown bits in the result.

Unary Reduction Operators

not	&
or	
xor	^
nand	~&
nor	~
xnor	^^ ^^

- Reduction operators perform a bit-wise operation on all the bits of a single operand.
- The result is always 1'b0, 1'b1 or 1'bX.

```

module reduction;
  localparam [3:0] CONST_A = 4'b0100,
                  CONST_B = 4'b1111;

  reg val;

  initial
  begin
    val = &CONST_A ;           // 0
    val = |CONST_A ;           // 1
    val = &CONST_B ;           // 1
    val = |CONST_B ;           // 1
    val = ^CONST_A ;           // 1
    val = ^CONST_B ;           // 0
    val = ~|CONST_A ;          // 0
    val = ~&CONST_A ;          // 1
    val = ^CONST_A & &CONST_B ; // 1
  end

endmodule

```

Unary reduction operators operate on all bits of a single operand to produce a single-bit result. The effect is as if they first applied the logical operation to the first two bits of the operand, then iteratively applied the logical operation to the current partial result and the next bit. The result of the operation is always a single bit that is 0, 1, or unknown (x).

You will see these same operators also used as bitwise binary operators. When used with a single operand, they are reduction operators.

Logical Operators

not	!
and	&&
or	

Logical operators interpret their operands as either true (1'b0) or false (1'b1) or unknown (1'bX).

0 – If all bits 0

1 – If any bit 1

X – If any bit is Z or X and no bit is 1

```
module logical;
  localparam integer FIVE = 5;
  localparam [3:0] CONST_A = 4'b0011,
                  CONST_B = 4'b10xz,
                  CONST_C = 4'b0z0x;

  reg ans;

  initial
  begin
    ans = !CONST_A;           // 0
    ans = CONST_A && 0;       // 0
    ans = CONST_A || 0;       // 1
    ans = CONST_A && FIVE;    // 1
    ans = CONST_A && CONST_B; // 1
    ans = CONST_C || 0;       // X
  end

endmodule
```



Logical and bitwise operators are identical for 1-bit expressions, but very different for vectors.

Logical operators reduce each operand to a single bit, and then perform a single bit operation.

The rules for reduction of an operand are as follows:

- If the operand contains all zeroes, it reduces to logic 0.
- If the operand contains any ones, it reduces to logic 1.
- If the operand contains no ones, but does contain one or more high-impedance or unknown values, it reduces to unknown, because its logical value is unknown.

The unary logical negation operator then inverts the logical sense of its operand. 0 becomes 1 and 1 becomes 0.

The binary conjunction and disjunction operators produce the logical conjunction and disjunction, respectively, of their operands.

Design Tip: Generally logical operators should be avoided unless explicitly required. It is very easy to get into the habit of using logical operators everywhere. They are identical to bitwise operators for single bit expressions, but give very different results when using vectors.

Arithmetic Operators

Verilog-1995:

Add	+
Subtract	-
Multiply	*
Divide	/
Modulus	%
Unsigned	Based literal, net, reg , time
Signed	Unbased literal, integer

Verilog-2001:

- Exponential Power **

```
module arithops;
  localparam integer CONST_INT = -3,
                CONST_5   = 5;
  localparam [3:0] rega = 3,
                regb = 4'b1010,
                regc = 14;

  integer val;
  reg [3:0] num;

  initial
  begin
    val = CONST_5 * CONST_INT; // -15
    val = (CONST_INT + 5)/2;   // 1
    val = CONST_5/CONST_INT;  // -1
    num = rega + regb;         // 1101
    num = rega + 1;            // 0100
    num = CONST_INT;           // 1101
    num = regc % rega;          // 0010
  end
endmodule
```

The Verilog 1995 arithmetic operators are add, subtract, multiply, divide and modulus. Verilog 2001 added the power operator.

This example declares two *integer* parameters, three 4-bit vector *reg* parameters, an *integer* variable and a 4-bit vector *reg* variable, and performs some operations with them. See where integer division discards the fractional part. See where assigning 3 to an unsigned 4-bit vector *reg* keeps the same rightmost four bits but now interprets the value as +13.

Some additional information about arithmetic operators:

- Any z or x bit in either operand produces an unknown (x) result.
- Integer division discards any remaining fractional part.
- Division or modulo by 0 produces an unknown (x) result.
- Raising 0 to a negative power produces an unspecified result.
- Raising a negative value to a real power produces an unspecified result.

Enhanced Signed Arithmetic

Verilog 2001

New reserved word: `signed`

- `signed` keyword treats literal, function, net, reg as signed.

```
32'shFDB97531
function signed [31:0] alu;
wire signed [15:0] addr;
reg signed [31:0] data;
```

- Arithmetic right shift operators maintain the sign of a value.
- `$signed` and `$unsigned` system functions cast value.



The parser interprets an integer with no base specifier as a signed value in 2's complement form.

- `intA = -12 / 3 ;`
- The left operand is a 32-bit signed value 12, negated.
- The expression result is `-4`.

The parser interprets an integer with an unsigned base specifier as an unsigned value.

- `intA = -'d12 / 3 ;`
- The left operand is a 32-bit unsigned value 12, negated.
- The expression result is 1431655761.

The parser interprets an integer with a signed base specifier as a signed value.

- `intA = -4'sd12 / 3 ;`
- The left operand is a 4-bit pattern of 12 (1100) interpreted as a signed number (`-4`) and then negated (`4`).
- The expression result is 1.

Signed Vectors

signed defines vector types as signed quantities.

- 2's complement:
 - Also for function returns.
 - Signed arithmetic for vectors is easier.
 - Assignments between `signed reg` and `integer` types maintain sign information.
 - Vector types can also be defined as unsigned.
- By default vectors are unsigned.

```
reg [3:0] usreg;  
// 4-bit vector in range 0 to 15  
reg signed [3:0] sreg;  
// 4-bit vector in range -8 to 7
```

```
function signed [15:0] add_2_signals  
...  
endfunction
```

```
module smult (input signed [7:0] opa, opb;  
              output signed [15:0] product);  
  
    assign product = opa * opb;  
  
endmodule
```

```
reg signed [3:0] areg;  
integer aint = -5;  
  
initial begin  
    areg = aint; // areg = -5  
    aint = areg; // aint = -5  
...  
end
```



For Verilog-2001, you can declare signed vectors. Vectors are by default unsigned. An unsigned vector is treated as signed for accesses through a signed port.

Shift Operators

Verilog-1995:

- **logical shift** << >>
 - Ignores operand signs.
 - Fills extra bits with 0.
 - Implements division or multiplication by powers of two.

Verilog-2001:

- **arithmetic shift** <<< >>>
 - Ignores right operand sign.
 - Left shift operates like logical left shift.
 - Right shift preserves the left operand sign if the result is a signed expression.

```
module shift;
  reg [7:0] rega = 8'b10011001;
  reg signed [7:0] regs = 8'b10011001;
  reg [7:0] regb;

  initial
  begin
    regb = rega << 1; // 00110010
    regb = rega >> 1; // 01001100
    regb = regs <<< 1; // 00110010
    regb = regs >>> 1; // 11001100
    regb = rega << -1; // 00000000
  end
endmodule
```

-1 is 2**32-1

The logical shift operators shift the left operand by the number of bit positions given by the right operand interpreted as a positive number, filling vacated bit positions with 0. You can use the logical shift operators to implement integer division or multiplication by powers of 2.

Verilog 2001 added the arithmetic shift operators. The arithmetic left shift operator operates exactly as does the logical left shift operator. The arithmetic right shift operator preserves the sign bit if the resulting expression is signed.

The example initializes an 8-bit vector rega to value 153 (8'h99). Left-shifting this value once is equivalent to doubling its value, and placing the result back into an 8-bit vector reg. The data truncates and the value gets back down to 50 (8'h32). Right-shifting rega value once is equivalent to halving its value and losing the fractional part, so produces 76 (8'h4c). Left-shifting by -1 is equivalent to left-shifting a very large number of times, so produces 0.

Relational Operators

less than	<
greater than	>
less than or equal to	<=
greater than or equal to	>=

The result is:

- 1'b0 if the relation is false.
- 1'b1 if the relation is true.
- 1'bX if either operand contains any z or x bits.

```
module relationals;
  reg [3:0] rega, regb, regc;
  reg val;

  initial
  begin
    rega = 4'b0011;
    regb = 4'b1010;
    regc = 4'b0x10;

    val = regc > rega ; // val = X
    val = regb < rega ; // val = 0
    val = regb >= rega; // val = 1
    val = regb > regc ; // val = X
  end
endmodule
```



If at least one operand is unsigned, the comparison treats both operands as unsigned and zero-extends a smaller operand to match the width of a wider operand.

If both operands are signed integer quantities, the operation sign-extends a smaller operand to match the width of a wider operand and treats the comparison as a signed integer comparison.

If at least one operand is real, the operation if necessary converts the other operand to real and treats the comparison as a real comparison.

The result is 0 if the relation is false and 1 if the relation is true. Any high-impedance (z) or unknown (x) bit in either operand produces an unknown result regardless of the truth of the relation.

Logical Equality and Case Equality Operators

logical equality ==

- Result can be unknown.

	0	1	Z	X
0	1	0	X	X
1	0	1	X	X
Z	X	X	X	X
X	X	X	X	X

```
...
a = 2'b1x;
b = 2'b1x;

if (a == b)
    // values match & do not contain Z or X
else
    // values do not match or contain Z or X
    // above values execute this else branch
```

case equality ===

- Result is always known.

	0	1	Z	X
0	1	0	0	0
1	0	1	0	0
Z	0	0	1	0
X	0	0	0	1



Called "case equality" because the case statement matches items this way.

```
...
a = 2'b1x;
b = 2'b1x;

if (a === b)
    // values match exactly
    // above values execute this if branch
else
    // values do not match
```

The difference between logical equality and case equality is the handling of high-impedance and unknown values. The logical equality treats high-impedance and unknown bits as truly unknown bit values, while the case equality treats high-impedance and unknown bits as values to be matched. A case equality operation thus always produces a 0 or 1 result and never an unknown result.

The case equality operator gets its name from the fact that the case statement, which you will see later, matches items in the same manner. Some people informally call it the “identity” operator because it checks that the bits are identical instead of that their values are the same.

More About Equality Operators

logical equality	==
logical inequality	!=
case equality	===
case inequality	!==

- For logical equalities, the result is always 1'b0, 1'b1 or 1'bX.
- For case equalities, the result is always 1'b0 or 1'b1.

```

module equalities;
  reg [3:0] rega, regb, regc;
  reg val;

  initial
  begin
    rega = 4'b0011;
    regb = 4'b1010;
    regc = 4'b1x10;

    val = rega == regb;    // val = 0
    val = rega != regb;    // val = 1
    val = regb != regc;    // val = X
    val = regc == regc;    // val = X
    val = rega === regb;   // val = 0
    val = rega !== regc;   // val = 1
    val = regb === regc;   // val = 0
    val = regc === regc;   // val = 1
  end
endmodule

```

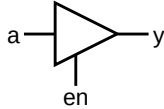


Here is another example illustrating the difference between the logical equality operators and the case equality operators.

The difference between logical equality and case equality is the handling of high-impedance (z) and unknown (x) values. The logical equality treats high-impedance and unknown bits as truly unknown bit values, while the case equality treats high-impedance and unknown bits as values to be matched. A case equality operation thus always produces a 0 or 1 result and never an unknown result.

Conditional Operator

Conditional ? :



Sometimes the conditional operator is more readable than the `if` statement. Sometimes not...

```
module tribuf1 (a, en, y);  
  input a, en;  
  output y;  
  assign y = en ? a : 1'bZ;  
endmodule
```

```
module tribuf2 (a, en, y);  
  input a, en;  
  output reg y;  
  always @(a or en)  
    y = en ? a : 1'bZ;  
endmodule
```

```
module tribuf3 (a, en, y);  
  input a, en;  
  output reg y;  
  always @(a or en)  
    if (en)  
      y = a;  
    else  
      y = 1'bZ;  
endmodule
```

The conditional operator is a ternary operator. It has three operands.

- If the first operand, the condition expression, is 1, then the operation result is the value of the second operand.
- If the first operand is 0, then the operation result is the value of the third operand.
- If the first operand is unknown (x), then the operation result is the value of the second operand merged with the value of the third operand so that if any bit position has the same value in both operands, then that bit position of the result also has that value.

So the conditional operator selects between two operands like a 2-to-1 multiplexor.

The operands can be any expression. It is very common to nest conditional operations.

Concatenation Operator

concatenation{ }

- Can select and join bits from different vectors to form a new vector
 - Forms unsigned expression
- Can reorganize vector bits to form a new vector
 - Endian swaps / reverse / rotate
- Can use on either side of an assignment!



Literals in a concatenation must be explicitly sized so that all the bits go into the correct position.

```
module concatenation;
  reg [7:0] rega, regb, regc, regd, new;
  reg [3:0] nib1, nib2;

  initial
  begin
    rega = 8'b00000011;
    regb = 8'b00000100;
    regc = 8'b00011000;
    regd = 8'b11100000;

    new = {regd[6:5], regc[4:3], regb[3:0]};
    // new = 8'b11_11_0100

    new = {2'b11, regb[7:5], rega[4:3], 1'b1};
    // new = 8'b11_000_00_1

    new = {regd[4:0], regd[7:5]};
    // rotate regd right 3 places
    // new = 8'b00000_111

    {nib1, nib2} = rega;
    // nib1 = 4'0000, nib2 = 4'0011

  end
endmodule
```

A concatenation operation joins together the bits of one or more comma-separated operands. You must size literal constant operands so that the operation can determine exactly where to place each bit.

Here are some examples that fail to size their operands:

```
a[7:0] = {5'b01010, 2}; //decimal value 2 unsized
c[3:0] = {3'b011, 'b0}; //binary value 'b0 unsized
```

Replication Operator

Replication { {} }

- Reproduces a concatenation a set number of times
- Syntax:
 - `{const_expr {sized_expr}}`
 - The constant number of repetitions must not have Z or X values.

```
module replicate;
  reg      rega = 1'b1;
  reg [1:0] regb = 2'b11;
  reg [3:0] regc = 4'b1001;
  reg [7:0] bus;

  initial
  begin

    // single bit rega replicated 8 times
    bus = {8{rega}};
    // bus = 11111111

    // 4x rega concatenated with 2x regc[1:0]
    bus = { {4{rega}}, {2{regc[1:0]}} };
    // bus = 1111_01_01

    // regc concatenated with 2x regb
    bus = { regc, {2{regb}} };
    // bus = 1001_11_11

    // regc concatenated with 2x 1'b1
    // and replicated 2 times
    bus = { 2{regc[2:1]}, {2{1'b1}} };
    // bus = 00_1_1_00_1_1

  end
endmodule
```

15 © Cadence Design Systems, Inc. All rights reserved.

cadence

A replication operation replicates a concatenation a nonnegative constant number of times. A replication operation applies only to a concatenation. You will not see it in any other context. The replication count cannot have any high-impedance (z) or unknown (x) values.

This example:

- 1st – Replicates the value of the single-bit rega variable value eight times to form an 8-bit value it assigns to the bus variable;
- 2nd – Concatenates four replications of the value of the single-bit rega variable with two replications of the value of the lowest two bits of the regc variable to form an 8-bit value it assigns to the bus variable;
- 3rd – Concatenates the value of the four-bit regc variable with two replications of the value of the two-bit regb variable to form an 8-bit value it assigns to the bus variable; and
- 4th – Concatenates the value of the middle two bits of the regc variable with two replications of the sized literal constant 1'b1, and replicates that 4-bit concatenation twice to form an 8-bit value it assigns to the bus variable.

Reference: Operator Precedence High to Low

Category	Symbol(s)
Unary	+ - ! ~ & ~& ~ ^ ~^ ^~
Exponential	**
Arithmetic	* / %
	+ - (binary)
Shift	<< >> <<< >>>
Relational	< <= > >=
Equality	== != === !==
Bit-wise	& (binary)
	^ ^~ ~^ (binary)
	(binary)
Logical	&&
Conditional	? :
Concatenation / Replication	{ } { { } }
Reliance on operator precedence may make your code unreadable – use parentheses!	

16 © Cadence Design Systems, Inc. All rights reserved.



Your reliance on operator precedence can make your code unreadable to the great majority of people that do not memorize this table. Your co-workers will appreciate your appropriate use of parenthesized expressions.

Quickly now, without looking at this table, insert parentheses to indicate the grouping of these expressions...

a ? ~ b * c << d : e < f ^ g || h

Give up? Don't you wish it was already parenthesized for you?

a ? ((~b * c) << d) : ((e < f) ^ g) || h)

Module Summary

This module presented the following Verilog operators:

- Reduction unary operators that perform a bit-wise operation across all bits of a single operand
- Arithmetic binary operators that preserve signage
- Shift binary operators – the right shift optionally preserves signage
- Relational binary operators
- Logical equality and case equality binary operators
- Bit-wise binary operators that perform individual logical operations on each bit position of two vectors
- Logical unary operator and binary operators that treat their operands as logical values
- Conditional ternary operator that selects between two operands based on the logical value of a third operand
- Concatenation operator that concatenates scalar or vector operands to create a new vector
- Replication operator that replicates a concatenation a fixed number of times



You can now appropriately and effectively use Verilog operators. To do that, you need to know what operators are available and what they do. This module provided an explanation and example for each Verilog operator.

Module Review

1. How wide is the result of a logical `&&` operation?
2. Explain the difference between the `&&` and `&` operators.
3. TRUE or FALSE: You must explicitly size literals in a concatenation.
4. Given the statement `regx = 4'b0101;` what is the value of:
`bus = { 2 {regx[3:1]}, {3{1'b0,regx[0]}}}` ;



This page does not contain notes.

Module Review Solutions

1. How wide is the result of a logical `&&` operation?
 - The logical `&&` operator reduces each operand to a single bit (1'b0, 1'b1 or 1'bx), then ANDs these bits together to give a single bit result.
2. Explain the difference between the `&&` and `&` operators.
 - The logical `&&` operator reduces each operand to a single bit, then ANDs these bits to give a single bit result. The bitwise `&` operator does a bit-by-bit AND of its operands, from LSB to MSB, producing a result which is the same width as the longest operand.
3. TRUE or FALSE: You must explicitly size literals in a concatenation.
 - TRUE. All operands in a concatenation must have a size. For example: `{3{b1}}` is illegal because 'b1 is not sized, while `{3{1'b1}}` is legal.
4. Given the statement `regx = 4'b0101;` what is the value of:
`bus = { 2 {regx[3:1], {3{1'b0,regx[0]}}} };`
 - Bus = 18'b010_01_01_01_01_01_01_01;



This page does not contain notes.



Module 11-2

Making Procedural Statements

cādence®

This module describes the Verilog procedural programming statements.

Module Objective

In this module, you:

- Describe design behavior procedurally

Topics

- Procedural blocks review
- Making procedural assignments
- Making conditional statements
- Making case statements
- Making loop statements

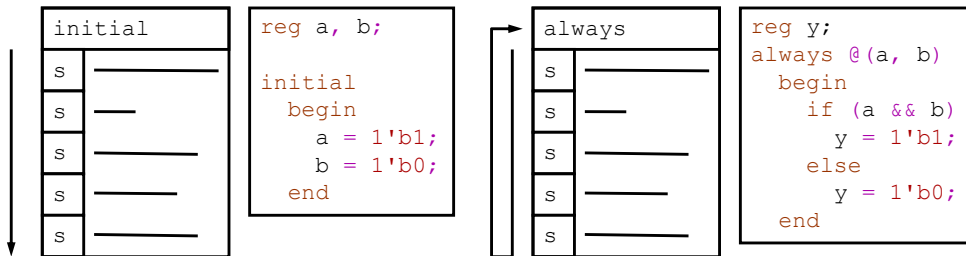


Your objective is to appropriately and effectively procedurally describe design behavior. To do that, you need to know generally about procedural blocks and procedural statements, and more specifically about the branching procedural statements. The Verilog behavioral modeling constructs are very similar to statements of the C programming language. They differ in subtle ways that can sometimes be irksome.

Describing Module Behavior

The **initial** construct starts execution at the start of simulation, and when complete, terminates the process.

The **always** construct starts execution at the start of simulation, and when complete, executes again.



22 © Cadence Design Systems, Inc. All rights reserved.

cadence

Event simulation relies upon processes. A process is an object that reacts to input events and generates output events. The continuous assignment that you have already seen is a process – the simulator reacts to its input transitions to calculate a new output value and transitions the net to that new value.

Verilog has other kinds of processes – two most obvious to the user are the **initial** construct and the **always** construct, frequently referred to as the initial block and the always block. The **initial** and **always** keywords are not statements themselves – they apply to their following statement and make it execute as a process. Their following statement is usually a statement block, e.g., statements between the **begin** and **end** keywords, but does not have to be.

- The simulator executes each **initial** process at the start of simulation, and upon completing its execution, terminates the process.
- The simulator executes each **always** process at the start of simulation, and upon completing its execution, executes it again.

Synchronizing Module Behaviors

- Procedures “block” upon encountering an event control.
- Procedures “unblock” upon occurrence of any of the listed events.
- An event control can reference:
 - A single event identifier.
 - An event expression.
 - The * wildcard operator.

*Examples

```
@a
@(a)
@(a or b)
@(a, b)
@*
@(*)
@(posedge clk)
@(posedge clk or negedge rstn)
```

Event
Expression

```
always @( a or b or sel )
  if (sel == 1)
    op = b;
  else
    op = a;
```

If an **always** construct has no construct to block its execution, then when the simulator started executing it at the start of simulation, execution would continue in a tight loop forever and the simulator would appear to hang.

Verilog provides procedural timing controls for “stepping” execution of procedural blocks. The most common of these is the event control. An event control starts with the **at (@)** character and then follows with either a wildcard character, a single event identifier, or a parenthesized event expression. The event expression can be a list of event expressions separated by the **or** keyword or by the comma (,) character. Both are shown here. The **or** keyword in an event expression is a separator between event expressions and is not an operator in the usual sense. You can further qualify an expression with the **posedge** or **negedge** keywords.

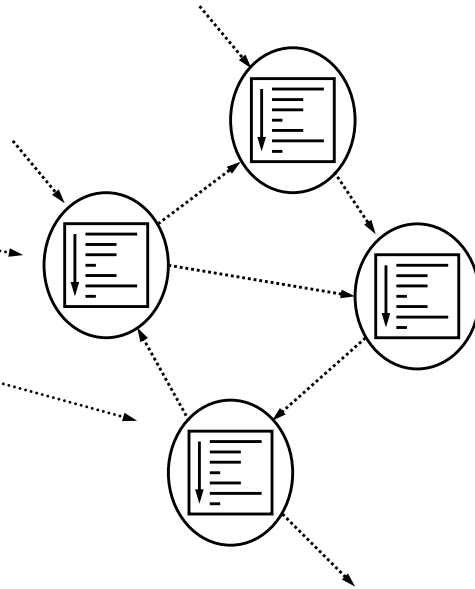
A timing control is not itself a statement, but precedes a statement, and in the case of an event control, blocks execution of that statement and subsequent sequential statements until one of those events occurs.

```
event_control ::=
  @ event_identifier
| @ ( event_expression )
| @*
| @ (*)

event_expression ::=
  expression
| hierarchical_identifier
| posedge expression
| negedge expression
| event_expression or event_expression
| event_expression , event_expression
```

Interactions Between Behaviors

```
module design (  
  list_of_port_declarations  
);  
  net_declaration(s)  
  reg_declaration(s)  
  
  always @*  
  begin  
    statement(s)  
  end  
  
  always @(posedge clk)  
  begin  
    statement(s)  
  end  
  
  assign  
    net_assignment(s)  
  
endmodule
```



A non-trivial design usually contains several modules, and a module at the RT level typically contains procedural blocks:

- Each sequential procedural block executes its statements sequentially like a conventional programming language;
- Multiple blocks execute concurrently, like hardware; and
- Multiple blocks communicate with each other using events, nets and variables.

This capability to have many procedural blocks communicating concurrently with each other is the basic model which Verilog uses to describe hardware.

Making Procedural Assignments

- Procedural assignments are assignments made inside procedures.
- You make procedural assignments only to variables.

```
module fulladder (  
    input wire a, b, cin,  
    output reg [1:0] out  
);  
  
    reg sum, carry;  
  
    always @(a, b, cin)  
    begin  
        sum  = a ^ b ^ cin;  
        carry = (a & b) | cin & (a ^ b);  
        out  = {carry, sum};  
    end  
  
endmodule
```



The ***initial*** construct and the ***always*** construct are frequently referred to as ***procedural blocks***, though as you have seen, they can apply to an atomic statement. Statements within a *procedural block* are procedural statements, and most of your procedural statements will be assignments.

- Recall that a continuous assignment is to a net and is its own process, so only exists ***outside*** any procedural blocks.
- A procedural assignment is to a variable and is not its own process, so only exists ***inside*** a procedural block.
- The right operand of either assignment can contain constants, variables and nets, as on the right side only their values are used.

Making Conditional Statements

- **if** is a conditional statement.
- **if** tests the conditions in sequence.
 - Condition is Boolean expression.
- Conditions can overlap.
- Executes statement associated with first known true condition.
- Statement can be null by just using a semicolon ;.

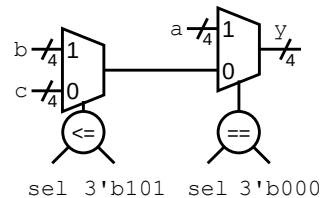
```

module if_example (
  input      [3:0] a, b, c,
  input      [2:0] sel,
  output reg [3:0] y
);

always @(a or b or c or sel)
  if (sel == 3'b000)
    y = a;
  else if (sel <= 3'b101)
    y = b;
  else
    y = c;

endmodule

```



26 © Cadence Design Systems, Inc. All rights reserved.

cadence

if statement syntax:

```

if (expression) statement
[ { else if (expression) statement } ]
[ else statement ]

```

The conditional statement is a two-way branch. If the conditional expression evaluates to a known non-zero value then the first branch executes and otherwise the second branch executes if it exists. The conditional expression can be multiple bits.

As with other programming languages, in nested conditional statements, every **else** keyword is associated with the nearest previous **if** keyword that does not already have an **else** associated with it. If this association displeases you, then you can force association by using **begin-end** sequential blocks, or you can just toss in the **else** keyword where needed with a null following statement.

In this example, execution of the procedural block blocks at the event control until a transition occurs on at least one of the module inputs. When a transition occurs, the conditional statement executes. It evaluates the *sel* signal. If the value is exactly 3'b000, then it executes the first branch, which is an assignment of *a* to *y*. If the value is anything else, it executes the second branch, which is another conditional statement. This conditional statement again evaluates the *sel* signal. If the value is less than or equal to 3'b101, then it executes the first branch. If the value is anything else, it executes the second branch. Note that such a chain of conditional statements implies priority. If the *sel* signal is exactly 3'b000, then the second test for less than 3'b101 is not made, even though the value would match that second test if it were made.

Conditional Statement Syntax

```
if (expression)
    statement
```

if branch executes if
condition is known true.

```
if (expression)
    statement
else
    statement
```

if branch executes if
condition is known true.
else branch executes if
condition is false or unknown.

```
if (expression)
    statement
else if (expression)
    statement
else
    statement
```

if branch executes if
condition is known true.
else if branch executes for
alternative condition known true.
else branch executes if
no condition known true.



The 1st illustration omits the second branch.

The 2nd illustration includes the second branch.

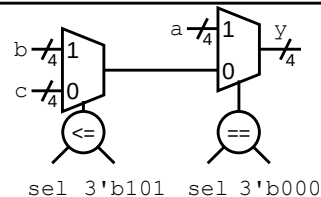
The 3rd illustration is not a new syntax. Unlike some languages, Verilog does not have an *elsif* keyword, so this is simply chaining two conditional statements.

Making Case Statements: case

- The **case** statement is a multiway prioritized branching statement.
- **case** item match expressions can overlap.
- Compares item match expressions to case expression in the order that they appear.
 - Uses case equality comparison
 - Bitwise 0,1,Z,X comparison
 - Executes statement associated with first match
 - Executes optional default item if no other item matches
 - In this example, inputs containing z or x

```
module case_example (
  input  [3:0] a, b, c,
  input  [2:0] sel,
  output reg [3:0] y
);

always @(a or b or c or sel)
  case (sel)
    0      : y = a;
    1,2,3,4,5 : y = b;
    6,7    : y = c;
    default : y = 'bx;
  endcase
endmodule
```



28 © Cadence Design Systems, Inc. All rights reserved.

cadence

case statement syntax:

```
case ( expression )
  expression { , expression } : statement
  {expression { , expression } : statement }
  [ default [:] statement ]
endcase
```

The **case** statement is a multiway prioritized branching statement. The case statement evaluates an expression and then evaluates a sequence of match expressions and executes the statement associated with only the first matching expression. The case statement considers high-impedance (z) and unknown (x) values and matches expressions that have a high-impedance or unknown bit in the same position in both expressions. You can comma-separate multiple match expressions associated with the same following statement. The following statement is often a statement block but does not have to be. You can optionally provide a **default** match item to be matched when no other match expression matches. It is common to place the default match item at the case statement end, but you can place it anywhere. You can have at most one default match item in each **case** statement.

In this example, execution of the procedural block is blocked at the event control until a transition occurs on at least one of the module inputs. When a transition occurs, the **case** statement executes. It evaluates the “sel” signal. If the value is 0 then it executes the first branch. If the value is between 1 and 5 then it executes the second branch. If the value is between 6 and 7 then it executes the third branch. If the value is anything else, it executes the default branch, which sets the output unknown. As an unknown value does not infer any particular hardware, the hardware has no implementation of this default branch.

Making Case Statements: `casex`

- Treats Z, X and ? characters as “don't care” bit positions in either:
 - case expression (`sel`)
 - case item expression `3'b0XX`
- Lets you group match values for more concise description
 - Encoders, decoders, etc.
 - Not very useful in testbenches
 - Not a preferred method usually
- `casex` treats X in case expression as meaningful in uninitialized logic
 - Hides initialization problems
 - Not recommended

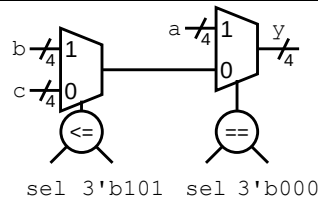
```
module casex_example (
    input      [3:0] a, b, c,
    input      [2:0] sel,
    output reg [3:0] y
);

always @(a or b or c or sel)
    casex (sel)
        3'b000      : y = a;
        3'b0??, 3'b?0? : y = b;
        3'b11X      : y = c;
        default      : y = 'bx;
    endcase
endmodule
```

Using ? is more readable than using X or Z



Can't match unknown or high impedance values coming out of DUT, so is less useful in testbenches.



29 © Cadence Design Systems, Inc. All rights reserved.

cadence

The ***casex*** statement is a variation of the *case* statement that lets you specify bit positions it should not compare.

The *casex* statement treats the x, z, and question mark (?) characters as *don't-care* bit values. Recall that in a literal constant the ? character is equivalent to the z character and that only the based literal constants can use the x, z, and ? characters.

The *casex* statement does not compare bit positions of either the case expression or the case match items that have any of these *don't-care* values.

By convention, you might want to use the ? character to indicate *don't-care* bit positions in literal constants to avoid confusion with the high-impedance and unknown values. However, the *casex* statement still considers bit positions containing high-impedance (z) or unknown (x) values in expressions of nets and variables to be positions it should not match, so a *casex* statement in a testbench cannot test high-impedance or unknown values coming from the DUT.

This example is functionally identical to the previous example, but uses *don't-care* bit positions to represent a range of matching values.

Making Case Statements: `casez`

- Treats `z` and `?` characters as “don't care” bit positions in:
 - case expression (`sel`)
 - case item expression `3'b0ZZ`
- Performs definitive match for `x`
 - Useful in testbenches!
- `casez` is preferable to `casex`.
 - `x` in case expression is not wildcard
 - Safer for uninitialized RTL code



Can match unknown values so is more useful in testbenches.

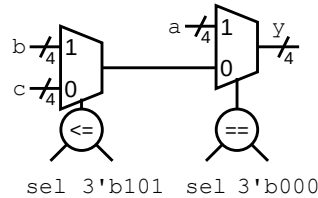


Use `?` instead of `z` – it's more intuitive and will cause less confusion!

```
module casez_example (
  input      [3:0] a, b, c,
  input      [2:0] sel,
  output reg [3:0] y
);

always @(a or b or c or sel)
  casez (sel)
    3'b000      : y = a;
    3'b0??, 3'b?0? : y = b;
    3'b11Z      : y = c;
    default     : y = 'bx;
  endcase
endmodule
```

Use `?` To avoid confusion with high impedance logic state `Z`



The **`casez`** statement is a variation of the **`casex`** statement and treats only the `z`, and question mark (`?`) characters as *don't-care* bit values and performs a definitive match for unknown (`x`) values. In a testbench, to test the existence of unknown values from the DUT, you should use the `casez` statement instead of the `casex` statement.

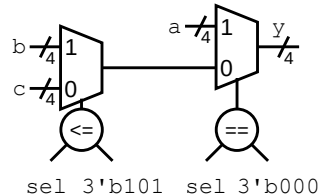
This example is functionally identical to the previous example, but uses the `z` character instead of the `x` character to represent the *don't-care* bit positions.

Alternate Case Statement Form

- **case** item match can be an expression as well.
- Case item match expression does not have to be a constant value!
- Can use case statement to match *variable* expressions.
- Case items are prioritized according to the order.

```
module finalcase (
  input      [3:0] a, b, c,
  input      [2:0] sel,
  output reg [3:0] y
);

always @(a or b or c or d)
  case (1'b1)
    sel == 3'b000 : y = a;
    sel <= 3'b101 : y = b;
    sel >= 3'b110 : y = c;
    default      : y = 'bx;
  endcase
endmodule
```



31 © Cadence Design Systems, Inc. All rights reserved.

cadence

Alternate **case** statement syntax:

```
case ( expression )
  expression { , expression } : statement
  {expression { , expression } : statement }
  [ default [:] statement ]
endcase
```

Unlike most programming languages, the Verilog case item expression does not have to be constant.

This example is functionally identical to the previous example, but tests which match expression first matches the value 1, that is, which one is the first “true” expression.

```
case (1'b1)
  en_a : op = a;
  en_b : op = b;
  en_c : op = c;
endcase
```

In the above code, the case statement en_a has a highest priority. The statement checks for en_a first. If this is 1'b1, then the output op is a. Otherwise it checks for en_b. If en_b is 1'b1 then output op = b. When both en_a and en_b not 1'b1, then en_c is checked. If en_c is 1'b1 then output op = c.

Making Loop Statements: **while**

- **while** loop iterates and executes its following statement while the expression is known and non-zero
- Must previously declare any variables used in expression
- While expression is known and true, executes statement

```
integer count;
...
while (count < 10)
begin
    // statements
    count = count + 1;
end
```

```
reg [7:0] tempreg;
reg [3:0] count;
...
// Count the ones in tempreg
count = 0;
while (tempreg)
begin
    if ( tempreg[0] )
        count = count + 1;
    tempreg = tempreg >> 1;
end
```

32 © Cadence Design Systems, Inc. All rights reserved.



while statement syntax:

```
while ( expression )
    statement
```

The **while** loop executes its following statement while the expression is known and non-zero. The following statement is usually a statement block but does not have to be. If the expression is not known or is zero, then the following statement never executes.

The 1st illustration executes a statement block while the count is known and less than 10. The last statement of the block increments the count. Some statement before the loop presumably initializes the count, but this is not shown.

The 2nd illustration counts the number of bits in the *tempreg* that are 1. While the “tempreg” is known and non-zero, if the least significant bit of the *tempreg* is known and non-zero it increments the count. In any case, each loop iteration shifts the *tempreg* one position to the right.

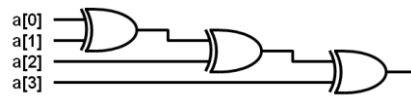
Making Loop Statements: `for`

- `for` loop requires prior declaration of any variables used in expressions.
- *Initialization* is done for the loop.
- Condition checked.
- *while* condition is known true, executes statement(s).
- *Update* expression is then executed at end of each iteration.

```
module parity (
  input [3:0] a,
  output reg odd
);

integer i;
always @*
begin
  odd = 0;
  for (i = 0; i <= 3; i = i + 1)
    odd = odd ^ a[i];
end

endmodule
```



33 © Cadence Design Systems, Inc. All rights reserved.

cadence

`for` loop Syntax:

```
for ( initializer exp; condition exp;
      update exp)
  statement;
```

You can more concisely rewrite a *while* loop as a *for* loop that is not as complex. The *for* loop performs the initial variable assignment and then while the test expression is known and non-zero executes the following statement. The following statement is usually a statement block but does not have to be. After each iteration of its following statement, the *for* loop executes the second variable assignment. The two variable assignments are almost always to a variable that participates in the test expression, but they don't have to be.

This example calculates the parity of an input vector `reg`. The *for* loop initializes an index to 0 and while the index is less than or equal to 3 it exclusive-ORs the partial result with each successively indexed bit of the input, each time incrementing the index by 1.

If you think for a moment, you can think of an operator that will do this in one assignment without a loop. The same *for* loop can also be implemented using an equivalent *while* loop. The syntax is:

```
Initializer exp;
while ( condition exp)
    statement;
update exp;
```

Making Loop Statements: **repeat**

- **repeat** loop executes the statements number of times specified in the loop.
- Must previously declare any variables used in expression
- For expressed count executes statement.
- The **repeat** loop is equivalent to a **for** loop with the index variable automatically declared, initialized and incremented.

```
module multiplier (
    input      [3:0] a, b,
    output reg [7:0] result
);

reg [7:0] temp_a;
reg [3:0] temp_b;

always @(a, b)
begin
    temp_a = a;
    temp_b = b;
    result = 0;
    repeat ( 4 )
    begin
        if ( temp_b[0] )
            result = result + temp_a;
        temp_a = temp_a << 1; // left
        temp_b = temp_b >> 1; // right
    end
end

endmodule
```

34 © Cadence Design Systems, Inc. All rights reserved.

cadence

repeat loop syntax:

```
repeat ( expression ) statement
```

You can more concisely rewrite a for loop as a *repeat* loop, where you do not need to use the index, for example, to select a bit of a vector. A *repeat* loop executes its following statement the number of times specified by its parenthesized expression.

This example multiplies two 4-bit inputs to produce an 8-bit output. Because it uses a shift multiplier implementation, it does not need a loop index, so uses a *repeat* loop instead of a *for* loop. Equivalent logic can be obtained using for loop as well. The syntax is:

```
integer h; // hidden index
for ( h=0; h < expression; h=h+1 ) statement
```

Making Loop Statements: **forever**

- **forever** loop executes statement forever.

```
reg clock;  
initial  
  forever  
    #10 clock = !clock ;
```



forever loop syntax:

forever statement

Verilog also provides a *forever* loop for when you want to repeat execution of a statement an essentially infinite number of times or until simulation ends. The same logic can be modeled using while loop as well. The syntax is:

```
while ( 1 )  
  statement
```

Module Summary

Now you can appropriately and correctly describe design behavior procedurally.

In this module, you studied these topics:

- The **initial** and **always** constructs introduce procedural blocks.
- The **@** event control blocks further procedural block execution until an event occurs.
- The **=** procedural assignment that is inside a procedural block you make only to variables.
- The **if-else**, and **case/casex/casez** procedural branching statements.
- The **for**, **while**, **repeat**, and **forever** procedural looping statements.



You should now be able to appropriately and effectively procedurally describe design behavior. This module described Verilog procedural blocks and procedural programming statements.

Module Exercise 1

```
// Find at least twelve things wrong!

module twelve_wrong (w, v, y, t, b, c, s)
input  [3:0] a, b, c, s;
output [3:0] w, v, y, t;
reg    [3:0] w, v, y, t,

assign W = s;

always @(a or b or c and s)
  if (s == 0)
    v = a;
    y = b;
    t = c;
  else if (s <= 4'b0101)
  begin
    v = c;
    y = b;
    t = a;
    a = s;
  end
  else if (s == 6 or s == 7)
    v = c;
  else v == 4'b'xxxx;
endmodule
```



This page does not contain notes.

Module Exercise 1 Solution

```
module twelve_wrong (w, v, y, t, b, c, s) // Append semicolon
input  [3:0] a, b, c, s;                // Include "a" in port list
output [3:0] w, v, y, t;                // Change first "," to ":"
reg    [3:0] w, v, y, t;                // Change last "," to ";"

assign W = s;                           // Change "W" to "w"
                                         // Change to procedural assignment
                                         // Change "and" to "or"
always @(a or b or c and s)
    if (s == 0)                          // Insert "begin"
        v = a;
        y = b;
        t = c;
    else if (s <= 4'b0101)                // Insert "end"
    begin
        v = c;
        y = b;
        t = a;
        a = s;                           // Change "a" to "w"
    end
    else if (s == 6 or s == 7)            // Change "or" to "||"
        v = c;
    else v == 4'b'xxxx;                  // Change "==" to "="
                                         // Remove second "'"
endmodule
```

This page does not contain notes.